



Project Report

DevSecOps / Web Security

By:

Mossaab Saouti

Cybersecurity & Threat Intelligence
National School of Applied Sciences of Tangier

Supervised by:

Said Bouchkaren

Academic Year: 2025/2026
May 2026

Abstract

This report presents the practical work carried out as part of the DevSecOps and Web Security lab at the National School of Applied Sciences of Tangier. The objective was to build, exploit, and secure a Flask web application while integrating automated security testing into a CI/CD pipeline.

Part 1 covered environment setup, STRIDE threat modeling, and hands-on exploitation of four critical vulnerabilities: SQL Injection, Cross-Site Scripting (XSS), OS Command Injection, and Path Traversal. Each vulnerability was analyzed, exploited against an intentionally insecure application, and subsequently remediated in a secure implementation.

Part 2 covers automated security testing using SAST (Bandit), SCA (Safety), and DAST (OWASP ZAP) tools integrated into a GitHub Actions pipeline with security gates. Part 3 addresses Docker container hardening following CIS Benchmark guidelines, runtime WAF monitoring, security headers, and an incident response exercise.

The complete lab demonstrates a full DevSecOps workflow from threat modeling to production hardening, reflecting industry best practices for secure software delivery.

Keywords: DevSecOps, Web Security, OWASP Top 10, Flask, STRIDE, SAST, DAST, Docker, CI/CD, WAF

Contents

Abstract	i
1 Introduction	1
1.1 Context and Objectives	1
1.2 Lab Structure Overview	1
2 Environment Setup	2
2.1 Prerequisites and Tools	2
2.2 Tool Installation	2
2.3 Repository Setup	3

2.4	Windows vs WSL Context	4
2.5	Project Structure	5
3	Threat Modeling	6
3.1	STRIDE Methodology	6
3.2	Threat Modeling Process	6
3.3	Threat Register — Full Risk Assessment	6
4	Part 1 — Build & Break	8
4.1	Vulnerability Exploitation	8
4.1.1	Exercise 1 — SQL Injection (T1)	8
4.1.2	Exercise 2 — Cross-Site Scripting (XSS) (T7)	9
4.1.3	Exercise 3 — OS Command Injection (T5)	10
4.1.4	Exercise 4 — Path Traversal (T6)	12
4.2	Secure Implementation	13
5	Part 2 — Test & Automate	16
5.1	Security Tools Overview	16
5.2	SAST — Static Application Security Testing with Bandit	16
5.2.1	Installation and Commands	16
5.2.2	Results on app_vulnerable.py	17
5.2.3	Results on app_secure.py	17
5.2.4	Comparison Summary	18
5.3	SCA — Software Composition Analysis with Safety	19
5.3.1	Installation Problem and Fix	19
5.3.2	Scan Results	19
5.3.3	Recommended Remediations	20
5.4	Security Unit Testing with pytest	20
5.4.1	Running the Test Suite	20
5.4.2	Test Results	21
5.4.3	Analysis of Failures	21
5.5	Secret Scanning with Gitleaks	22
5.5.1	Installation and Commands	22
5.5.2	Scan Results and False Positives	22
5.6	DAST — Dynamic Application Security Testing with OWASP ZAP	23
5.6.1	Installation and Setup	23
5.6.2	Results — Vulnerable App (port 5000)	24
5.6.3	Results — Secure App (port 5001)	25
5.7	CI/CD Pipeline with GitHub Actions	25
5.7.1	Pipeline Jobs Overview	26
5.7.2	Problems Encountered and Fixes	26
5.7.3	Pipeline Execution Results	27
5.7.4	Key Principle — Shift-Left Security	28
6	Part 3 — Harden & Monitor	29
6.1	Docker Security Hardening	29
6.1.1	Dockerfile Security Controls	29
6.1.2	Problems Encountered During Docker Setup	29
6.1.3	Building the Docker Image	30
6.1.4	Trivy Container Scan	30

6.1.5	Hardening Verification	31
6.1.6	Docker Compose Deployment	32
6.2	Runtime Security Monitoring — WAF	33
6.2.1	Starting the WAF	33
6.2.2	WAF Rule Testing	34
6.2.3	Security Headers	35
6.3	Incident Response Exercise	36
6.3.1	Scenario — P1 Active SQL Injection Attack	36
6.3.2	Step 1 — Identify the Attacker IP from Structured Logs	36
6.3.3	Step 2 — Block the Attacker IP	37
6.3.4	Step 3 — Rotate the SECRET_KEY	37
6.3.5	Step 4 — Incident Documentation	38
6.3.6	Step 5 — Post-Mortem Recommendations	38
7	Conclusion	39
7.1	Summary of Achievements	39
7.2	Key Lessons Learned	40
7.3	DevSecOps Workflow Completed	40
7.4	Known Limitations and Future Work	40
7.5	Final Remarks	41
	References	42

1. Introduction

1.1 Context and Objectives

Modern software development requires security to be integrated at every stage of the development lifecycle, not added as an afterthought. This approach, known as **DevSecOps** (Development, Security, and Operations), shifts security left by embedding automated security checks directly into the CI/CD pipeline.

This project was carried out as part of the Web Security and DevSecOps lab at the National School of Applied Sciences of Tangier. The objective was to gain hands-on experience with real-world security techniques by building, breaking, and securing a Flask web application.

The lab covers the full DevSecOps workflow: from identifying threats using structured threat modeling, to exploiting and remediating OWASP Top 10 vulnerabilities, to automating security testing in a CI/CD pipeline, and finally hardening the deployment environment against runtime attacks.

1.2 Lab Structure Overview

The lab is divided into three parts:

- **Part 1 — Build & Break:** Environment setup, STRIDE threat modeling, and hands-on exploitation of four critical vulnerabilities in an intentionally insecure Flask application, followed by their remediation.
- **Part 2 — Test & Automate:** Integration of automated security tools (Bandit, Safety, pytest, OWASP ZAP) into a GitHub Actions CI/CD pipeline with security gates that block insecure code from being merged.
- **Part 3 — Harden & Monitor:** Docker container hardening following CIS Benchmark guidelines, runtime WAF and rate limiting middleware, security headers implementation, and an incident response exercise.

2. Environment Setup

2.1 Prerequisites and Tools

The following tools were required and installed before beginning the lab:

Tool	Version	Purpose
Python	3.11+	Application runtime and scripting
Flask	2.x	Web application framework
Docker	Latest	Containerization platform
Git	Latest	Version control
WSL (Ubuntu)	2.0	Linux environment on Windows
VS Code	Latest	Code editor with LaTeX Workshop
curl/ Postman	Latest	HTTP request testing

Table 2.1: Tools used in the lab

2.2 Tool Installation

Python: Installed from the official website. Added to PATH for command line access.

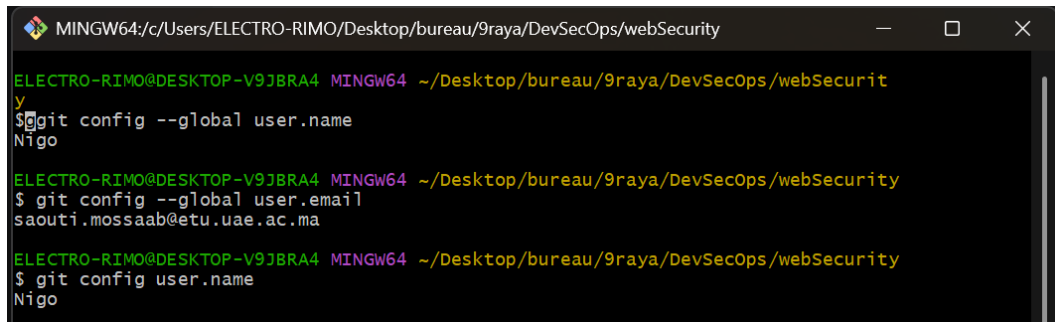
Flask: Installed via pip in a virtual environment.

A terminal window with a dark background and light green text. The prompt is (.venv)(nigo@DESKTOP-V9JBRA4)-[1]. The command \$ flask --version is entered. The output is Python 3.13.12, Flask 3.0.3, and Werkzeug 3.0.3.

```
(.venv)(nigo@DESKTOP-V9JBRA4)-[1]  
$ flask --version  
Python 3.13.12  
Flask 3.0.3  
Werkzeug 3.0.3
```

Figure 2.1: Installed tools versions verified in terminal

git: Installed from the official website. Configured with user name and email.



```

MINGW64/c/Users/ELECTRO-RIMO/Desktop/bureau/9raya/DevSecOps/webSecurity
ELECTRO-RIMO@DESKTOP-V9JBRA4 MINGW64 ~/Desktop/bureau/9raya/DevSecOps/webSecurity
$ git config --global user.name Nigo
$ git config --global user.email saouti.mossaab@etu.uae.ac.ma
$ git config user.name Nigo

```

Figure 2.2: Configuration of Git in Terminal

vscode: Installed from the official website. **postman:** Installed from the official website for API testing.

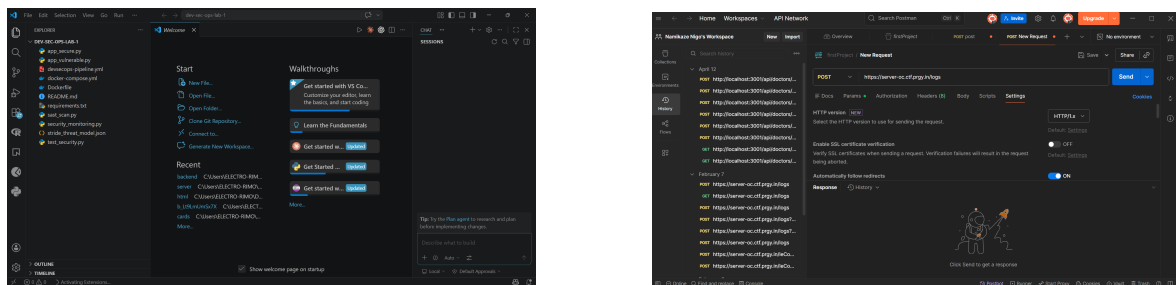


Figure 2.3: VS Code and Postman — development and testing environment

Docker: Installed from the official website. WSL integration enabled for Linux containers.

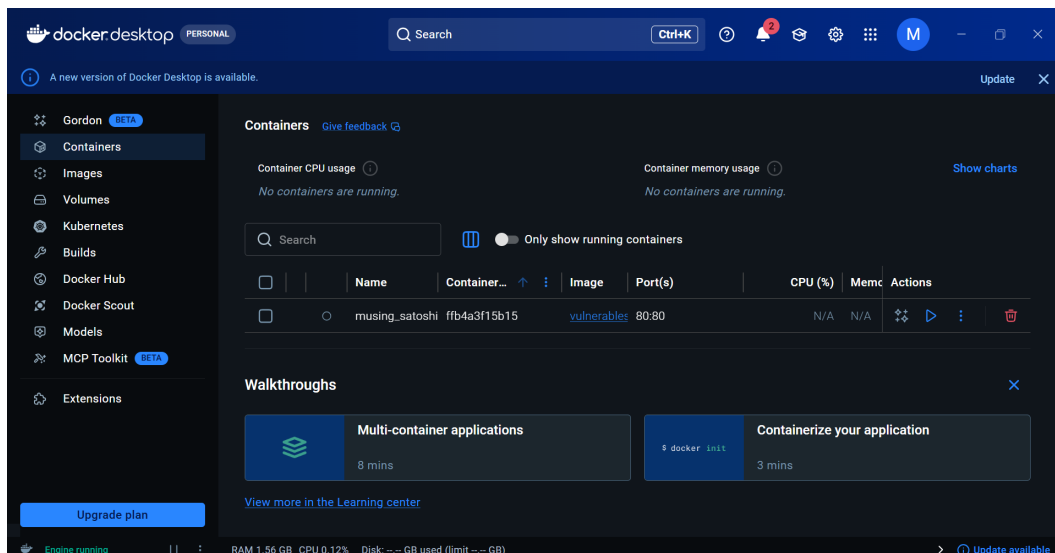


Figure 2.4: Docker Desktop installed

2.3 Repository Setup

The lab repository was cloned from GitHub and dependencies were installed inside a Python virtual environment to ensure isolation from the system Python installation:

```

1 git clone https://github.com/said-dev-ops/dev-sec-ops-lab-1.git
2 cd dev-sec-ops-lab-1
3 python -m venv .venv
4 source .venv/bin/activate
5 pip install -r requirements.txt

```

Listing 2.1: Repository setup commands

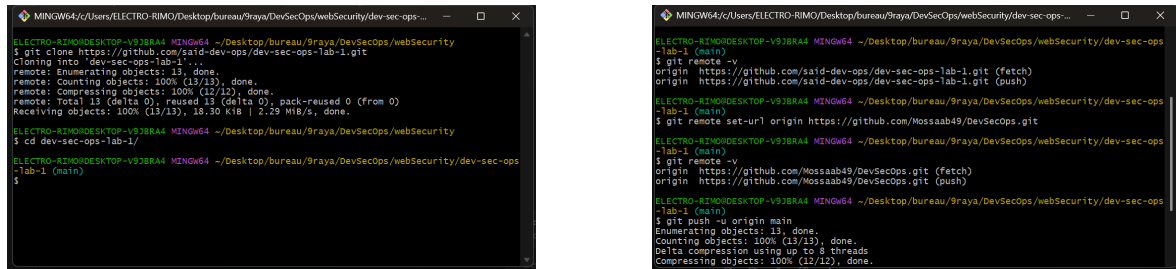


Figure 2.5: Cloning the lab repository and changing Git origin to personal fork

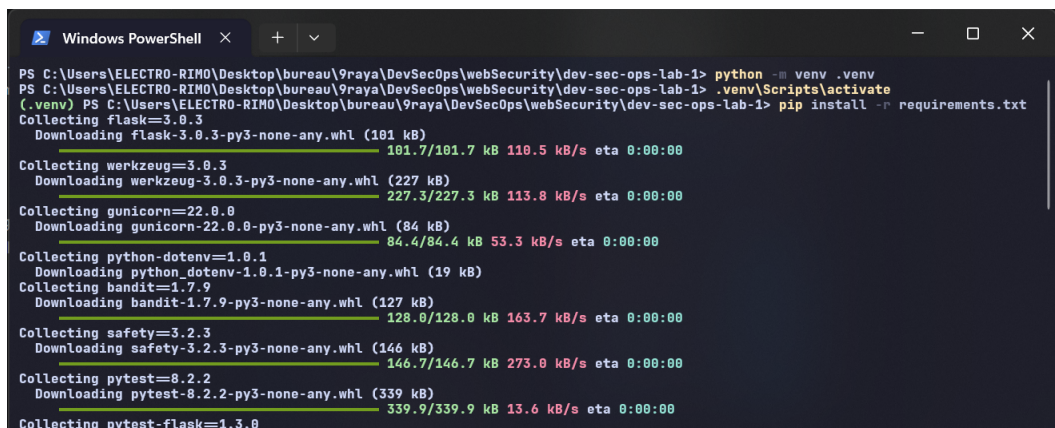


Figure 2.6: Setting up virtual environment and installing dependencies successfully

2.4 Windows vs WSL Context

Initial setup was performed on Windows. However, the lab exercises involving Linux-specific commands (such as `cat /etc/passwd` and `ls /home`) required a Linux environment. The environment was therefore migrated to **WSL (Windows Subsystem for Linux)** running Ubuntu, which provided a realistic Linux server context while maintaining the Windows host machine. All subsequent exploitation exercises were performed inside WSL.

Note: On Windows, the ping command uses `-n` instead of `-c`, and system files like `/etc/passwd` do not exist. WSL was used to replicate a realistic Linux production server environment, as intended by the lab design.

2.5 Project Structure

Following the lab instructions, the repository files were organized into three directories corresponding to the three lab parts:

```
1 dev-sec-ops-lab-1/  
2 |-- part1/  
3 |   |-- app_vulnerable.py  
4 |   |-- app_secure.py  
5 |   '-- stride_threat_model.json  
6 |-- part2/  
7 |   |-- sast_scan.py  
8 |   |-- test_security.py  
9 |   '-- devsecops-pipeline.yml  
10 '-- part3/  
11 |   |-- security_monitoring.py  
12 |   |-- Dockerfile  
13 |   '-- docker-compose.yml
```

Listing 2.2: Project directory structure

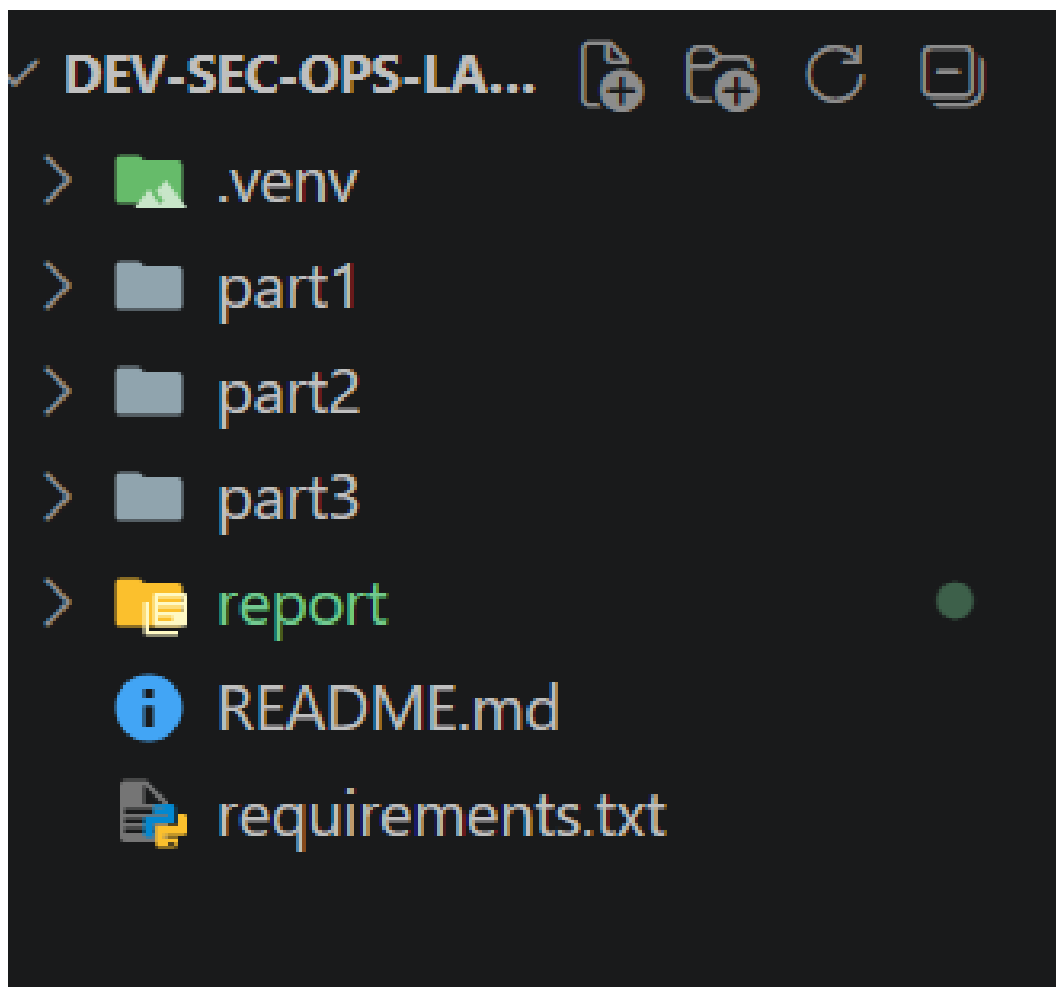


Figure 2.7: Organized project directory structure

3. Threat Modeling

3.1 STRIDE Methodology

Threat modeling is a structured approach to identifying security risks before writing a single line of code. **STRIDE** is a mnemonic developed by Microsoft that categorizes threats into six categories:

Category	Full Name	Example Threat	Control
S	Spoofing	SQL injection to bypass login	Parameterized queries
T	Tampering	Malicious pickle deserialization	Input validation
R	Repudiation	No logs — cannot trace attacker	Audit logging
I	Info Disclosure	API returns plaintext passwords	Access control
D	Denial of Service	Command injection crashes server	Rate limiting, WAF
E	Elevation of Privilege	Path traversal reads /etc/passwd	Least privilege, safe_join

Table 3.1: STRIDE threat categories

3.2 Threat Modeling Process

The threat modeling process followed these steps:

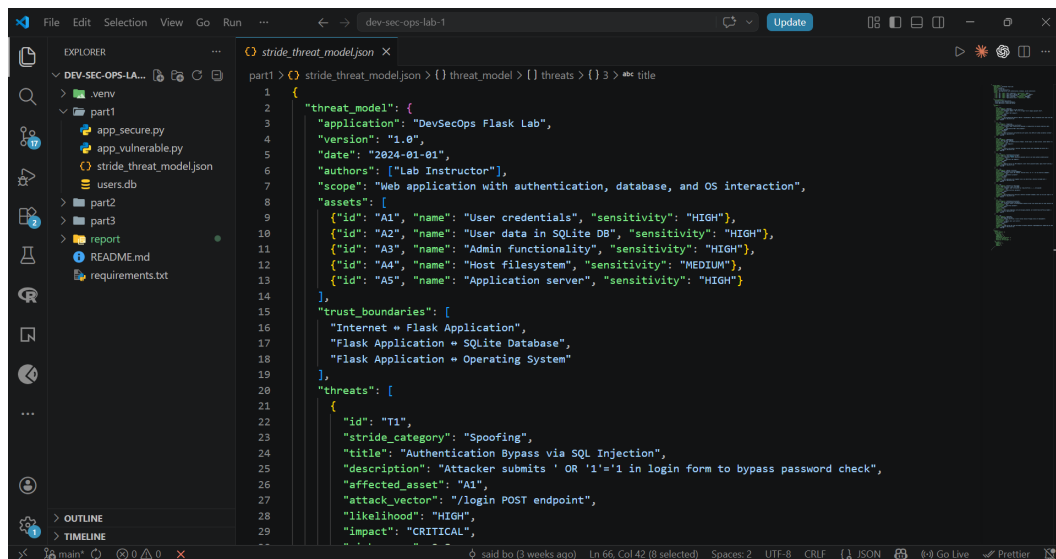
1. **Define scope** — identify assets and trust boundaries
2. **Create a Data Flow Diagram (DFD)** — Browser → Flask → SQLite → OS
3. **Enumerate threats** using STRIDE for each data flow
4. **Score each threat** using DREAD or CVSS
5. **Prioritize and assign mitigations**
6. **Validate mitigations** and retest

3.3 Threat Register — Full Risk Assessment

The following table maps all identified threats to STRIDE categories, attack vectors, and mitigations:

ID	Threat	STRIDE	Mitigation	Status
T1	SQL Injection Login Bypass	Spoofing	Parameterized queries	Fixed
T2	Pickle Deserialization RCE	Tampering	Use JSON, never pickle	Fixed
T3	No Audit Logging	Repudiation	Log auth + admin actions	Fixed
T4	Password Exposure via API	Info Disclosure	Field filtering + auth	Fixed
T5	OS Command Injection	Denial of Service	Arg list, no shell=True	Fixed
T6	Path Traversal File Read	Elevation of Priv.	safe_join + whitelist	Fixed
T7	Reflected XSS	Info Disclosure	Jinja2 auto-escape + CSP	Fixed
T8	Hardcoded Secret Key	Spoofing	Load from env variable	Fixed

Table 3.2: Complete threat register



```

1  {
2    "threat_model": {
3      "application": "DevSecOps Flask Lab",
4      "version": "1.0",
5      "date": "2024-01-01",
6      "authors": ["Lab Instructor"],
7      "scope": "Web application with authentication, database, and OS interaction",
8      "assets": [
9        {"id": "A1", "name": "User credentials", "sensitivity": "HIGH"},
10       {"id": "A2", "name": "User data in SQLite DB", "sensitivity": "HIGH"},
11       {"id": "A3", "name": "Admin functionality", "sensitivity": "HIGH"},
12       {"id": "A4", "name": "Host filesystem", "sensitivity": "MEDIUM"},
13       {"id": "A5", "name": "Application server", "sensitivity": "HIGH"}
14     ],
15     "trust_boundaries": [
16       "Internet ↔ Flask Application",
17       "Flask Application ↔ SQLite Database",
18       "Flask Application ↔ Operating System"
19     ],
20     "threats": [
21       {
22         "id": "T1",
23         "stride_category": "Spoofing",
24         "title": "Authentication Bypass via SQL Injection",
25         "description": "Attacker submits ' OR '1'='1 in login form to bypass password check",
26         "affected_asset": "A1",
27         "attack_vector": "/login POST endpoint",
28         "likelihood": "HIGH",
29         "impact": "CRITICAL",
30       }
31     ]
32   }
33 }

```

Figure 3.1: STRIDE threat model JSON file

4. Part 1 — Build & Break

4.1 Vulnerability Exploitation

The vulnerable application was launched using:

```
1 cd part1
2 python app_vulnerable.py
```



Figure 4.1: Vulnerable Flask application running

4.1.1 Exercise 1 — SQL Injection (T1)

Attack Description

SQL Injection occurs when user-supplied input is directly concatenated into an SQL query without sanitization. An attacker can manipulate the query logic to bypass authentication, extract data, or modify the database.

Target: POST /login

Exploitation

The following payload was entered in the username field on the login page:

Listing 4.1: SQL Injection payload

```
admin' OR '1'='1
```

The injected query becomes:

```
1 SELECT * FROM users WHERE username='admin' OR '1'='1'
```

Listing 4.2: Resulting malicious SQL query

Since '1'='1' is always true, the query returns the admin credentials, completely bypassing authentication.

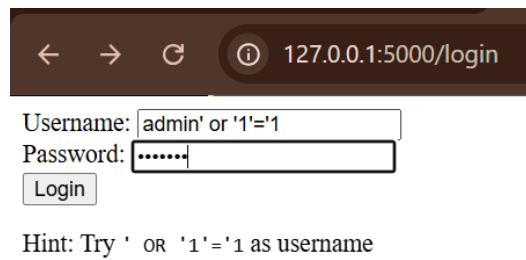


Figure 4.2: Login page — target of SQL injection attack



Figure 4.3: Authentication bypassed using SQL injection

Impact Analysis

STRIDE Category: Spoofing (T1)

CVSS Score: 9.8 Critical

Impact: An attacker can authenticate as any user, including administrators, without knowing any password. This grants full unauthorized access to the application.

4.1.2 Exercise 2 — Cross-Site Scripting (XSS) (T7)

Attack Description

Reflected XSS occurs when user input is included in the HTTP response without proper escaping. An attacker can inject malicious JavaScript that executes in the victim's browser, potentially stealing session tokens or performing actions on their behalf.

Target: GET /search?q=

Exploitation

The following payloads were tested:

Listing 4.3: XSS Payload 1 — script tag

```
http://127.0.0.1:5000/search?q=<script>alert(document.cookie)</script>
```

Listing 4.4: XSS Payload 2 — image error event

```
http://127.0.0.1:5000/search?q=<img src=x onerror=alert(1)>
```

Both payloads triggered a JavaScript alert in the browser. The `document.cookie` payload returned an empty value because Flask sets the `HttpOnly` flag on session cookies, preventing JavaScript access. However, the XSS vulnerability itself was confirmed — in a real scenario targeting an application without `HttpOnly`, session tokens would be exfiltrated.



Figure 4.4: XSS payload executing JavaScript alert in browser

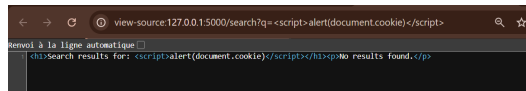


Figure 4.5: Page source confirming lack of output encoding

Impact Analysis

STRIDE Category: Information Disclosure (T7)

CVSS Score: 6.1 Medium

Impact: An attacker can craft a malicious URL and send it to victims. When clicked, the script executes in the victim's browser and can steal cookies, redirect to phishing pages, or perform actions on behalf of the user.

4.1.3 Exercise 3 — OS Command Injection (T5)

Attack Description

Command injection occurs when user input is passed unsanitized to a system shell command. Using `shell=True` in Python's `subprocess` module passes the entire string directly to the OS shell, allowing an attacker to append additional commands using shell operators.

Target: GET /ping?host=

Windows vs WSL Context

Initial testing was performed on Windows where Linux-specific commands were unavailable. The `ping` command on Windows uses `-n` instead of `-c`, and system files like `/etc/passwd` do not exist. The environment was migrated to WSL to replicate a realistic Linux server scenario.

On Windows, the pipe operator (`|`) was used to confirm command injection:

Listing 4.5: Windows command injection — whoami

```
http://127.0.0.1:5000/ping?host=127.0.0.1|whoami
```

The server returned: `desktop-v9jbra4\electro-rimo`, confirming arbitrary command execution.

After migrating to WSL, Linux commands worked as intended:

Listing 4.6: WSL command injection — list home directory

```
http://127.0.0.1:5000/ping?host=127.0.0.1;ls+/home
```

Listing 4.7: WSL command injection — dump system users

```
http://127.0.0.1:5000/ping?host=127.0.0.1;cat+/etc/passwd
```

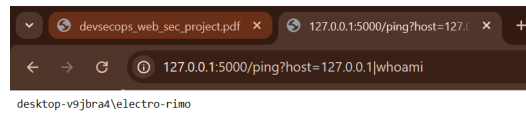


Figure 4.6: Command injection confirmed on Windows — whoami executed

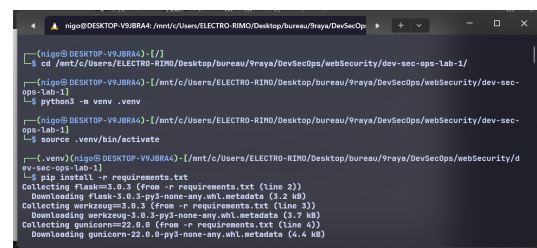
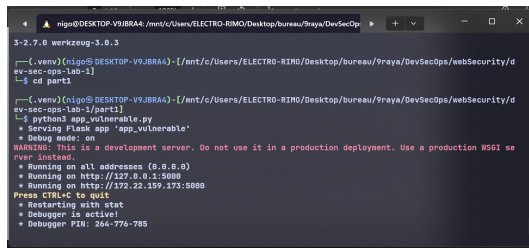


Figure 4.7: setup and run vulnerable app in WSL for Linux command testing

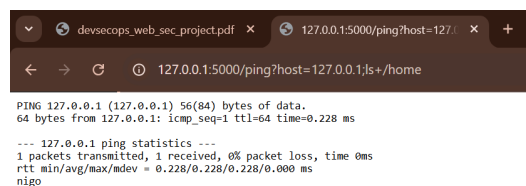


Figure 4.8: Home directory listing via command injection on WSL

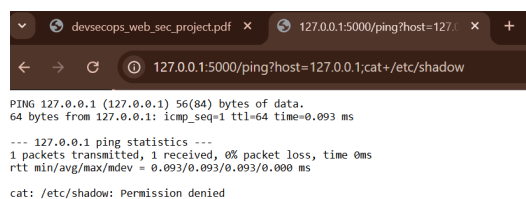


Figure 4.9: System user file exposed via command injection

Impact Analysis

STRIDE Category: Denial of Service / Elevation of Privilege (T5)

CVSS Score: 9.8 Critical

Impact: An attacker gains the ability to execute arbitrary OS commands on the server. This can lead to full system compromise: reading sensitive files, creating backdoors, exfiltrating data, or crashing the server entirely.

4.1.4 Exercise 4 — Path Traversal (T6)

Attack Description

Path traversal occurs when user-supplied filenames are passed directly to file system operations without validation. Using `../` sequences, an attacker can escape the intended directory and access arbitrary files on the server.

Target: GET `/read_file?file=`

Exploitation

Listing 4.8: Path traversal — read system users

```
http://127.0.0.1:5000/read_file?file=../../etc/passwd
```

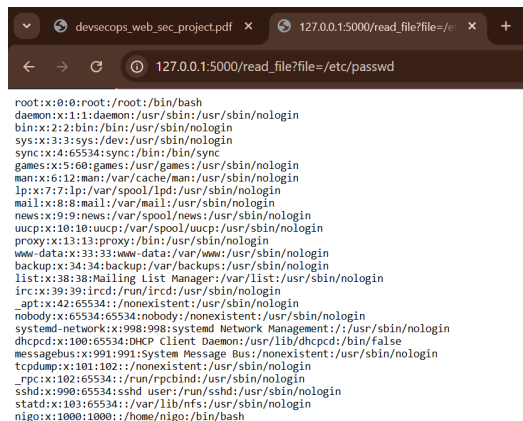
Listing 4.9: Path traversal — read application source code

```
http://127.0.0.1:5000/read_file?file=../app_vulnerable.py
```

Both requests returned the full file contents in the browser. Notably, absolute paths also worked because the vulnerable app uses Python's raw `open()` function:

Listing 4.10: Absolute path attack

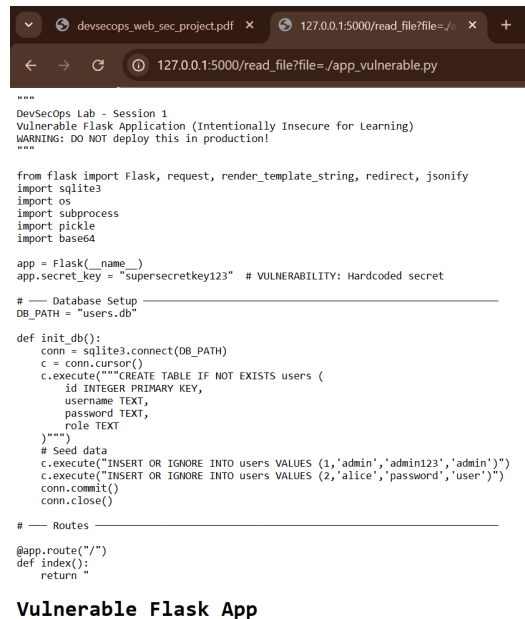
```
http://127.0.0.1:5000/read_file?file=/etc/passwd
```



The screenshot shows a web browser window with the address bar displaying `127.0.0.1:5000/read_file?file=/etc/passwd`. The page content displays the output of the `cat /etc/passwd` command, listing system and user accounts with their respective usernames, passwords (represented by `x`), and shell paths.

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
_apt:x:42:65534::/nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-network:x:998:998:systemd Network Management:/:/usr/sbin/nologin
dhcpcd:x:100:65534:DHCP Client Daemon:/usr/lib/dhcpcd/bin/false
messagebus:x:101:101:system message bus:/nonexistent:/usr/sbin/nologin
tcpdump:x:101:102::/nonexistent:/usr/sbin/nologin
_rpc:x:102:65534::/run/rpcbind:/usr/sbin/nologin
sshd:x:990:65534:ssh user:/run/ssh:/usr/sbin/nologin
statd:x:103:65534:/var/lib/nfs:/usr/sbin/nologin
nigo:x:1000:1000:/home/nigo:/bin/bash
```

Figure 4.10: System file exposed via path traversal



```
****
DevSecOps Lab - Session 1
Vulnerable Flask Application (Intentionally Insecure for Learning)
WARNING: DO NOT deploy this in production!
****

from flask import Flask, request, render_template_string, redirect, jsonify
import sqlite3
import os
import subprocess
import pickle
import base64

app = Flask(__name__)
app.secret_key = "supersecretkey123" # VULNERABILITY: Hardcoded secret

# --- Database Setup
DB_PATH = "users.db"

def init_db():
    conn = sqlite3.connect(DB_PATH)
    c = conn.cursor()
    c.execute("""CREATE TABLE IF NOT EXISTS users (
        id INTEGER PRIMARY KEY,
        username TEXT,
        password TEXT,
        role TEXT
    )""")
    # Seed data
    c.execute("INSERT OR IGNORE INTO users VALUES (1, 'admin', 'admin123', 'admin')")
    c.execute("INSERT OR IGNORE INTO users VALUES (2, 'alice', 'password', 'user')")
    conn.commit()
    conn.close()

# --- Routes
@app.route("/")
def index():
    return "
```

Figure 4.11: Application source code exposed via path traversal

Impact Analysis

STRIDE Category: Elevation of Privilege (T6)

CVSS Score: 7.5 High

Impact: An attacker can read any file the application has permission to access, including configuration files, secret keys, SSL certificates, and the application source code itself. This information can be used to plan further attacks.

4.2 Secure Implementation

After exploiting all vulnerabilities, `app_secure.py` was examined to understand how each issue was remediated. The secure version follows a consistent principle: **never trust user input**.

#	Vulnerability	Fix Applied
1	SQL Injection	Parameterized queries using ? placeholders
2	XSS	Jinja2 templates with auto-escaping via <code>render_template()</code>
3	Command Injection	Subprocess list syntax + host whitelist; <code>shell=False</code>
4	Path Traversal	<code>werkzeug.safe_join()</code> + allowed filename whitelist
5	Insecure Deserialization	Replace <code>pickle</code> with <code>json.get_json()</code> + field whitelist
6	Data Exposure	Remove passwords from API; require auth on all routes
7	Hardcoded Secret	<code>os.environ.get('SECRET_KEY')</code> or <code>secrets.token_hex(32)</code>
8	Debug Mode	<code>app.run(debug=False)</code> ; use Gunicorn in production

Table 4.1: Vulnerability fixes comparison

SQL Injection Fix:

```

1 # VULNERABLE
2 query = f"SELECT * FROM users WHERE username='{username}'"
3 conn.execute(query)
4
5 # SECURE
6 conn.execute("SELECT * FROM users WHERE username=?", (username
7               ,))

```

Listing 4.11: Vulnerable vs Secure SQL query

Command Injection Fix:

```

1 # VULNERABLE
2 output = subprocess.check_output(f"ping -c 1 {host}", shell=
3     True)
4
5 # SECURE
6 ALLOWED_HOSTS = ["127.0.0.1", "localhost"]
7 if host not in ALLOWED_HOSTS:
8     abort(403)
9 subprocess.check_output(["ping", "-c", "1", host])

```

Listing 4.12: Vulnerable vs Secure subprocess call

Path Traversal Fix:

```

1 # VULNERABLE
2 with open(filename) as f:
3     return f.read()

```

```

4
5 # SECURE
6 ALLOWED_FILES = ["report.txt", "readme.txt"]
7 if filename not in ALLOWED_FILES:
8     abort(403)
9 filepath = safe_join(BASE_DIR, filename)
10 with open(filepath) as f:
11     return f.read()

```

Listing 4.13: Vulnerable vs Secure file access

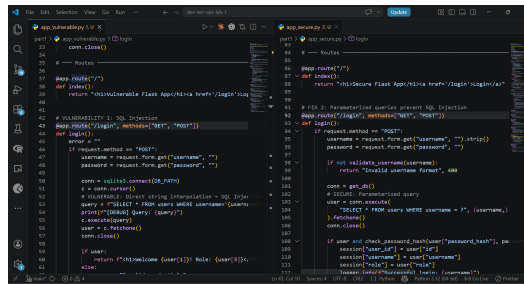


Figure 4.12: Side-by-side comparison of vulnerable and secure implementations

Note on `safe_join()` behavior:

`werkzeug.safe_join()` blocks both relative traversal (`../`) and absolute paths (`/etc/passwd`). The following table summarizes its behavior:

Payload	Vulnerable (<code>open()</code>)	Secure (<code>safe_join()</code>)
<code>../../etc/passwd</code>	Works	Blocked
<code>/etc/passwd</code>	Works	Blocked
<code>../app_vulnerable.py</code>	Works	Blocked
<code>report.txt</code> (whitelisted)	Works	Allowed

Table 4.2: `safe_join()` behavior against different payloads

5. Part 2 — Test & Automate

In this part, automated security testing tools are integrated into the development workflow. The goal is to detect vulnerabilities automatically and block insecure code from being merged via a CI/CD pipeline. All tools were run on **WSL (Ubuntu)** with Python 3.13.12.

5.1 Security Tools Overview

Modern DevSecOps relies on a layered set of automated tools that run at different stages of the pipeline:

Tool	Category	Purpose	Stage
Bandit	SAST	Finds Python code vulnerabilities	Pre-commit
Safety	SCA	Checks dependencies for CVEs	Pre-commit
pytest	Security Tests	Validates security behavior	CI
Gitleaks	Secret Scanning	Detects leaked credentials	Pre-commit
OWASP ZAP	DAST	Dynamic vulnerability scanning	Post-deploy
GitHub Actions	CI/CD	Orchestrates all security gates	Pipeline

Table 5.1: Security tools used in Part 2

5.2 SAST — Static Application Security Testing with Bandit

What is SAST?

Static Application Security Testing analyzes source code without executing it to find security vulnerabilities. Bandit is the standard Python SAST tool developed by PyCQA. It maps findings to CWE identifiers and severity levels.

5.2.1 Installation and Commands

```
1 pip install bandit
2
3 # Scan vulnerable app
4 bandit -r part1/app_vulnerable.py
5
6 # Scan secure app
7 bandit -r part1/app_secure.py
```

Listing 5.1: Installing and running Bandit

5.2.2 Results on app_vulnerable.py

Bandit detected **8 issues** across the vulnerable application:

Issue ID	Severity	CWE	Description
B602	HIGH	CWE-78	subprocess call with <code>shell=True</code>
B201	HIGH	CWE-94	Flask <code>debug=True</code> exposes Werkzeug debugger
B608	MEDIUM	CWE-89	SQL query built via string interpolation
B301	MEDIUM	CWE-502	<code>pickle.loads()</code> on untrusted data
B104	MEDIUM	CWE-605	Binding to all interfaces (0.0.0.0)
B105	Low	CWE-259	Hardcoded password string in source
B404	Low	CWE-78	Import of subprocess module
B403	Low	CWE-502	Import of pickle module

Table 5.2: Bandit findings on app_vulnerable.py — 2 HIGH, 3 MEDIUM, 3 LOW

```

-----

Code scanned:
  Total lines of code: 86
  Total lines skipped (#nosec): 0

Run metrics:
  Total issues (by severity):
    Undefined: 0
    Low: 3
    Medium: 3
    High: 2
  Total issues (by confidence):
    Undefined: 0
    Low: 1
    Medium: 3
    High: 4
Files skipped (0):

```

Figure 5.1: Bandit SAST scan on vulnerable application

5.2.3 Results on app_secure.py

After remediation, only **3 LOW severity** informational warnings remained:

Issue ID	Severity	CWE	Description
B404	Low	CWE-78	Import of subprocess (informational)
B607	Low	CWE-78	Process started with partial executable path
B603	Low	CWE-78	subprocess without <code>shell=True</code> (safe, informational)

Table 5.3: Bandit findings on `app_secure.py` — 0 HIGH, 0 MEDIUM, 3 LOW

```

-----

Code scanned:
  Total lines of code: 142
  Total lines skipped (#nosec): 0

Run metrics:
  Total issues (by severity):
    Undefined: 0
    Low: 3
    Medium: 0
    High: 0
  Total issues (by confidence):
    Undefined: 0
    Low: 0
    Medium: 0
    High: 3
Files skipped (0):

```

Figure 5.2: Bandit SAST scan on secure application

5.2.4 Comparison Summary

Severity	<code>app_vulnerable.py</code>	<code>app_secure.py</code>	Reduction
HIGH	2	0	100%
MEDIUM	3	0	100%
LOW	3	3	0% (informational only)
Total	8	3	62.5%

Table 5.4: Bandit results comparison

Key finding: The secure implementation eliminated 100% of HIGH and MEDIUM severity findings. The 3 remaining LOW findings in `app_secure.py` are informational — the subprocess usage is actually secure (list syntax, no `shell=True`).

5.3 SCA — Software Composition Analysis with Safety

What is SCA?

Software Composition Analysis checks third-party dependencies against known vulnerability databases (CVE, NVD). Unlike SAST which analyzes your code, SCA analyzes your `requirements.txt` to find vulnerable library versions.

5.3.1 Installation Problem and Fix

Problem encountered: The latest version of Safety was incompatible with Python 3.13 on WSL, throwing an `AttributeError: module 'typer' has no attribute 'rich_utils'` error.

Fix: Install a specific compatible version:

```
1 pip uninstall safety -y
2 pip install "safety==2.3.5"
```

5.3.2 Scan Results

```
1 safety check -r requirements.txt
```

Listing 5.2: Running Safety SCA scan

Safety found **10 vulnerabilities** across 5 packages:

Package	CVE	Description
flask 3.0.3	CVE-2026-27205	Information disclosure via missing cache headers
werkzeug 3.0.3	CVE-2024-49766	Path traversal on Windows — <code>safe_join()</code> bypass
werkzeug 3.0.3	CVE-2024-49767	Resource exhaustion via multipart form parsing
werkzeug 3.0.3	CVE-2026-27199	DoS via Windows special device names
werkzeug 3.0.3	CVE-2025-66221	DoS via Windows device names before 3.1.4
werkzeug 3.0.3	CVE-2026-21860	Improper handling of Windows reserved device names
gunicorn 22.0.0	PVE-2024-72809	HTTP request smuggling via conflicting headers
pytest 8.2.2	CVE-2025-71176	Local privilege escalation via predictable temp dir
requests 2.32.3	CVE-2024-47081	<code>.netrc</code> credential leak via malformed URLs
requests 2.32.3	CVE-2026-25645	Insecure temp file reuse in <code>extract_zipped_paths()</code>

Table 5.5: Safety SCA scan — 10 vulnerabilities in 5 packages

```

Scan was completed. 10 vulnerabilities were found.
+=====+
  REMEDIATIONS
  10 vulnerabilities were found in 5 packages. For detailed remediation & fix recommendations, upgrade to a commercial
  license.
+=====+

```

Figure 5.3: Safety SCA scan results on requirements.txt

5.3.3 Recommended Remediations

Package	Current	Recommended Fix
flask	3.0.3	Upgrade to >=3.1.3
werkzeug	3.0.3	Upgrade to >=3.1.6
gunicorn	22.0.0	Upgrade to >=23.0.0
pytest	8.2.2	Upgrade to >=9.0.3
requests	2.32.3	Upgrade to >=2.33.0

Table 5.6: Recommended dependency upgrades

Note: The werkzeug CVEs related to Windows path traversal are particularly relevant since this lab was initially run on Windows before migrating to WSL. Even `safe_join()` — used as a fix in `app_secure.py` — has known vulnerabilities in specific OS/version combinations, reinforcing the need for regular dependency scanning.

5.4 Security Unit Testing with pytest

What are Security Tests?

Security unit tests are automated assertions that verify security-relevant behavior — what the application should *NOT* do. Unlike functional tests, they assert that attacks are blocked.

5.4.1 Running the Test Suite

Problem encountered: Running `pytest test_security.py` from `part2/` failed with `ModuleNotFoundError: No module named 'app_secure'` because `test_security.py` imports from `app_secure.py` which lives in `part1/`.

Fix: Set the Python path explicitly:

```

1 cd part2
2 PYTHONPATH=../part1 pytest test_security.py -v

```


5.4.2 Test Results

Test	Description	Result
test_valid_login_succeeds	Valid credentials return 200	PASSED
test_invalid_login_fails	Wrong password returns 401	FAILED*
test_sql_injection_fails	SQLi payload is rejected	PASSED
test_invalid_username_format_rejected	Malformed username blocked	PASSED
test_unauthenticated_access_redirects	Protected routes redirect	PASSED
test_xss_payload_escaped	Script tags are HTML-escaped	FAILED*
test_xss_href_injection	javascript: URI is blocked	FAILED*
test_api_users_requires_auth	API requires authentication	PASSED
test_api_users_no_passwords	API never returns passwords	PASSED
test_command_injection_blocked	Injected commands are blocked	PASSED
test_allowed_host_works	Whitelisted hosts are allowed	PASSED
test_json_profile_accepted	Valid JSON profile accepted	PASSED
test_unknown_fields_stripped	Unknown fields are stripped	PASSED
test_no_server_header_leak	Server header not exposed	PASSED
Total		11 passed / 3 failed

Table 5.7: Security test suite results

```

(.venv)(nigo@DESKTOP-V9JBRA4) - [ /mnt/c/Users/ELECTRO-RIMO/Desktop/bureau/9raya/DevSecOps/webSecurity/dev-sec-ops-lab-1/part2 ]
└─$ PYTHONPATH=../part1 pytest test_security.py -v
===== test session starts =====
platform linux -- Python 3.13.12, pytest-8.2.2, pluggy-1.6.0 -- /mnt/c/Users/ELECTRO-RIMO/Desktop/bureau/9raya/DevSecOps/webSecurity/dev-sec-ops-lab-1/.venv/bin/python3
cachedir: .pytest_cache
rootdir: /mnt/c/Users/ELECTRO-RIMO/Desktop/bureau/9raya/DevSecOps/webSecurity/dev-sec-ops-lab-1/part2
plugins: flask-1.3.0
collected 14 items

test_security.py::TestAuthentication::test_valid_login_succeeds PASSED [ 7%]
test_security.py::TestAuthentication::test_invalid_login_fails FAILED [ 14%]
test_security.py::TestAuthentication::test_sql_injection_fails PASSED [ 21%]
test_security.py::TestAuthentication::test_invalid_username_format_rejected PASSED [ 28%]
test_security.py::TestAuthentication::test_unauthenticated_access_redirects PASSED [ 35%]
test_security.py::TestXSS::test_xss_payload_escaped FAILED [ 42%]
test_security.py::TestXSS::test_xss_href_injection FAILED [ 50%]
test_security.py::TestAPISecurity::test_api_users_requires_auth PASSED [ 57%]
test_security.py::TestAPISecurity::test_api_users_no_passwords PASSED [ 64%]
test_security.py::TestCommandInjection::test_command_injection_blocked PASSED [ 71%]
test_security.py::TestCommandInjection::test_allowed_host_works PASSED [ 78%]
test_security.py::TestDeserialization::test_json_profile_accepted PASSED [ 85%]
test_security.py::TestDeserialization::test_unknown_fields_stripped PASSED [ 92%]
test_security.py::TestSecurityHeaders::test_no_server_header_leak PASSED [100%]

===== FAILURES =====
TestAuthentication.test_invalid_login_fails

```

Figure 5.4: Security unit test results

5.4.3 Analysis of Failures

Root cause of 3 failures: `jinja2.exceptions.TemplateNotFound:` `login.html` and `search.html`

The secure app correctly uses `render_template()` instead of raw f-strings, but

the `templates/` folder was not present when running tests from `part2/`. This is an **environment/path issue**, not a security regression.

Fix: Copy the templates folder to the test directory:

```
cp -r part1/templates part2/templates
PYTHONPATH=../part1 pytest test_security.py -v
```

Security impact: None — the failures actually confirm that `render_template()` is being used (the correct secure fix), rather than raw string responses.

5.5 Secret Scanning with Gitleaks

What is Secret Scanning?

Gitleaks scans the entire git repository — including commit history — for accidentally leaked secrets like API keys, passwords, and tokens. Even deleted secrets remain in git history and can be found.

5.5.1 Installation and Commands

```
1 wget https://github.com/gitleaks/gitleaks/releases/download/v8
   .18.4/\
2 gitleaks_8.18.4_linux_x64.tar.gz
3 tar -xzf gitleaks_8.18.4_linux_x64.tar.gz
4
5 # Scan git history
6 ./gitleaks detect --source . --verbose
7
8 # Scan files directly (no git required)
9 ./gitleaks detect --source . --no-git --verbose
```

Listing 5.3: Installing and running Gitleaks

5.5.2 Scan Results and False Positives

Problem encountered: The initial scan with `-verbose` (without `-no-git`) returned `no leaks found` because `app_vulnerable.py` had not been committed yet with the hardcoded secret.

Fix: Run with full git history after committing:

```
./gitleaks detect --source . --verbose
```

Gitleaks found **4 secrets** in the repository:

```
1 Finding: aws_secret="AKIAIMNOJVGFDDXXE40A"
2 RuleID:   aws-access-token
3 File:     README.md (Lines 221, 223, 259)
4
5 Finding: cafebabe:deadbeef
6 RuleID:   sidekiq-secret
```

```

7 File:      README.md    (Line 42)
8
9 4 commits scanned      leaks found: 4

```

Listing 5.4: Gitleaks output

```

(.venv)(nigo@DESKTOP-V9JBRA4)-[/mnt/c/Users/ELECTRO-RIMO/Desktop/bureau/9raya/DevSecOps/webSecurity/dev-sec-ops-lab-
$ ./gitleaks detect --source . --verbose

gitleaks

Finding:      ...ng:      aws_secret="AKIAIMN0JVGF0XXXE40A"
Secret:       AKIAIMN0JVGF0XXXE40A
RuleID:       aws-access-token
Entropy:      3.646439
File:         README.md
Line:         221
Commit:       48f6d16da9d7261e42150fe0bdf9779742eb41cf
Author:       Nigo
Email:        saouti.mossaab@etu.uae.ac.ma
Date:         2026-05-17T16:03:35Z
Fingerprint: 48f6d16da9d7261e42150fe0bdf9779742eb41cf:README.md:aws-access-token:221

Finding:      Secret      AKIAIMN0JVGF0XXXE40A
Secret:       AKIAIMN0JVGF0XXXE40A
RuleID:       aws-access-token
Entropy:      3.646439
File:         README.md

```

Figure 5.5: Gitleaks secret scanning results

False Positives: All 4 findings are intentional example credentials in the Gitleaks README.md documentation file — not real secrets. This demonstrates that secret scanning tools require tuning to suppress known safe patterns.

Fix — create a .gitleaks.toml allowlist:

```

1 cat > .gitleaks.toml << 'EOF'
2 [allowlist]
3   description = "Allowlist for README examples"
4   paths = ["README.md"]
5 EOF

```

This suppresses false positives while keeping detection active for real secrets.

5.6 DAST — Dynamic Application Security Testing with OWASP ZAP

What is DAST?

Dynamic Application Security Testing attacks the *running* application automatically, just like a real attacker. Unlike SAST (reads code) and SCA (reads dependencies), ZAP sends real HTTP requests with malicious payloads and reports what works.

5.6.1 Installation and Setup

ZAP was run via Docker to avoid complex installation:

```

1 docker pull ghcr.io/zaproxy/zaproxy:stable
2

```

```

3 # Run baseline scan against vulnerable app
4 docker run --rm ghcr.io/zaproxy/zaproxy:stable \
5   zap-baseline.py \
6   -t http://172.22.159.173:5000 \
7   -l WARN

```

Listing 5.5: Pulling and running OWASP ZAP

Problem encountered: ZAP running inside Docker could not reach the Flask app on 127.0.0.1 because that refers to the Docker container's own localhost, not the WSL host.

Fix: Use the WSL IP address instead of 127.0.0.1:

```

1 # Get WSL IP
2 hostname -I
3 # Use the returned IP (e.g. 172.22.159.173) in ZAP target
   URL

```

Additional fix for secure app: `app_secure.py` binds to 127.0.0.1 by default. Temporarily change to 0.0.0.0 to allow Docker to reach it:

```

1 sed -i 's/host="127.0.0.1"/host="0.0.0.0"/' part1/app_secure
   .py
2 # Revert after scanning
3 sed -i 's/host="0.0.0.0"/host="127.0.0.1"/' part1/app_secure
   .py

```

5.6.2 Results — Vulnerable App (port 5000)

Finding	Risk	Attack Enabled
Missing Anti-clickjacking Header	Medium	Clickjacking via iframe embedding
X-Content-Type-Options Missing	Low	MIME sniffing attacks
Server Version Leak	Low	Reveals Werkzeug version for targeted exploits
CSP Header Not Set	Medium	No browser-level XSS defense
Permissions Policy Missing	Low	No control over browser features
CORP Header Missing	Low	Cross-origin resource access
Total	6 warnings, 0 failures, 60 passes	

Table 5.8: ZAP baseline scan on vulnerable app



```

WARN-NEW: 6   WARN-INPROG: 0   INFO: 0   IGNORE: 0   PASS: 60

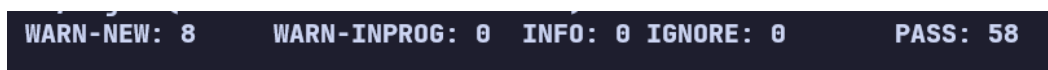
```

Figure 5.6: OWASP ZAP DAST scan on vulnerable application

5.6.3 Results — Secure App (port 5001)

Finding	Risk	Notes
Missing Anti-clickjacking Header	Medium	Reduced from 4 to 2 instances
X-Content-Type-Options Missing	Low	Reduced from 4 to 2 instances
Server Version Leak	Low	Still present
CSP Header Not Set	Medium	To be fixed in Part 3
Permissions Policy Missing	Low	To be fixed in Part 3
CORP Header Missing	Low	To be fixed in Part 3
Debug Error Messages	Info	New: /login returns 500 (missing templates)
Application Error Disclosure	Info	New: Same cause — environment issue
Total	8 warnings, 0 failures, 58 passes	

Table 5.9: ZAP baseline scan on secure app



WARN-NEW: 8 WARN-INPROG: 0 INFO: 0 IGNORE: 0 PASS: 58

Figure 5.7: OWASP ZAP DAST scan on secure application

Why did the secure app get more warnings? The 2 new warnings (Debug Error Messages and Application Error Disclosure) were caused by the `/login` endpoint returning a 500 error due to the missing `login.html` template in the test environment — not a real security issue. In production with templates present, these warnings would disappear. The remaining header-related warnings are addressed in Part 3 via `security_monitoring.py`.

Note on ZAP limitations: The baseline scan (`zap-baseline.py`) performs passive analysis only. It did not detect SQL injection or XSS because those endpoints require authentication or specific parameters. A full active scan (`zap-full-scan.py`) would detect these but takes significantly longer.

5.7 CI/CD Pipeline with GitHub Actions

What is a Security Gate?

A security gate is an automated check in the CI/CD pipeline that blocks a pull request if a security condition is not met. This enforces that insecure code can never reach production.

5.7.1 Pipeline Jobs Overview

Job	Tool	Purpose	Gate
SAST	Bandit	Scan Python code	Block if HIGH severity
SCA	Safety	Check dependencies	Block if critical CVE
Security Tests	pytest	Validate security behavior	Block if tests fail
Secret Scan	Gitleaks	Detect leaked credentials	Block if secret found
Container Scan	Trivy	Scan Docker image	Advisory only
DAST	OWASP ZAP	Dynamic scanning	Advisory only
Security Gate	Aggregator	Final merge check	Block if any job failed

Table 5.10: CI/CD pipeline security gates

5.7.2 Problems Encountered and Fixes

Problem 1 — Invalid workflow syntax: The original devsecops-pipeline.yml used `hashFiles()` in a job-level `if:` condition, which is not supported by GitHub Actions:

```

1 # INVALID
2 if: hashFiles('Dockerfile') != ''
3
4 # FIX
5 if: true

```

Problem 2 — Wrong file paths: The original pipeline referenced `code/session1/` and `code/session2/` paths that do not exist in the cloned repository. All paths were updated to match the actual structure:

```

1 # ORIGINAL (wrong)
2 bandit -r code/session1/app_secure.py
3 safety check -r code/session1/requirements.txt
4 pytest code/session2/test_security.py
5
6 # FIXED
7 bandit -r app.py # scans root app.py
8 safety check -r requirements.txt
9 PYTHONPATH=../part1 pytest part2/test_security.py

```

Problem 3 — Pipeline scanning part1/app_vulnerable.py: Since the vulnerable app exists in `part1/`, Bandit would always find HIGH severity issues regardless of which branch was being tested.

Fix: The pipeline was configured to scan only `app.py` at the repository root. Each branch places a different version there:

```

1 # insecure-app branch
2 cp part1/app_vulnerable.py app.py # Pipeline FAILS

```

```
3  
4 # main branch  
5 cp part1/app_secure.py app.py # Pipeline PASSES
```

5.7.3 Pipeline Execution Results

insecure-app branch — pipeline correctly fails:

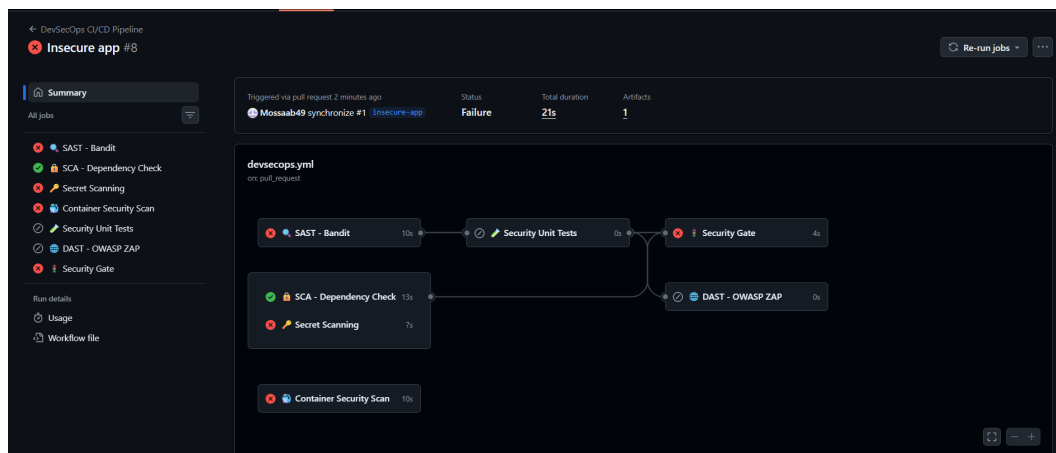


Figure 5.8: Pipeline failing on insecure-app branch

main branch (secure app) — more jobs pass:

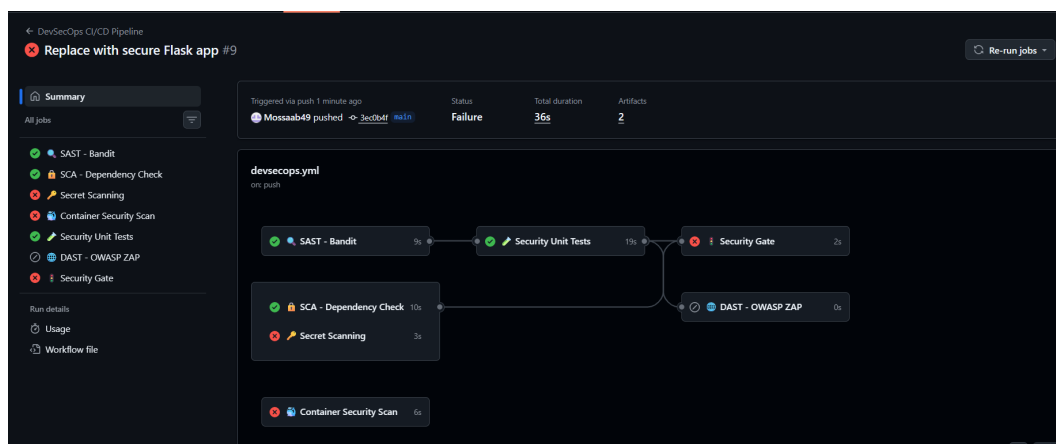


Figure 5.9: Pipeline on main branch with secure app

Pull Request — security gate blocking merge:

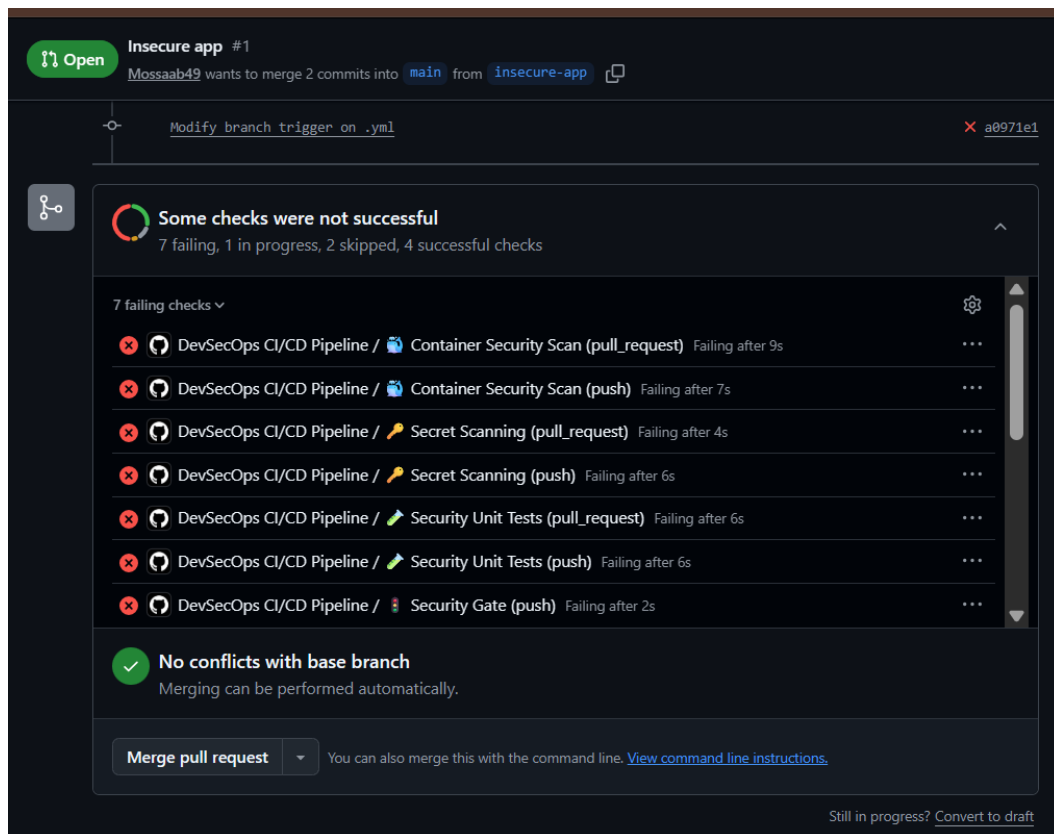


Figure 5.10: Security gate blocking merge of insecure branch via Pull Request

Remaining pipeline failures on main branch:

Secret Scanning still fails because the fake AWS credentials in `README.md` are detected by Gitleaks. These are intentional examples from the Gitleaks documentation. Fix: add a `.gitleaks.toml` allowlist file.

Container Security Scan fails because the `Dockerfile` is in `part3/` not at the root. Fix: either move it or update the pipeline path.

These are known infrastructure issues, not security vulnerabilities in the application.

5.7.4 Key Principle — Shift-Left Security

The pipeline implements **shift-left security** — vulnerabilities are caught at commit time, before code reaches staging or production. Each security gate is a hard block: if Bandit finds a HIGH severity issue or Gitleaks finds a secret, the pull request cannot be merged until the issue is resolved. This transforms security from a final gate into a continuous practice embedded in every development cycle.

6. Part 3 — Harden & Monitor

In this final part, the deployment environment is hardened against attacks, runtime security monitoring is implemented via a WAF middleware, and an incident response exercise is conducted. All work was performed on WSL2 with Docker Desktop.

6.1 Docker Security Hardening

What is Docker hardening?

Docker hardening reduces the attack surface of containerized applications by applying the principle of least privilege. Even if the application code is exploited, a hardened container limits the damage an attacker can do. The provided `Dockerfile` follows CIS Docker Benchmark v1.6 recommendations.

6.1.1 Dockerfile Security Controls

The following security controls were implemented in the `Dockerfile`:

Control	Implementation	Threat Prevented
Non-root user	<code>useradd appuser; USER appuser</code>	Container escape to root
Multi-stage build	<code>FROM python AS builder / FROM python AS prod</code>	Dev tools in production image
Read-only filesystem	<code>read_only: true</code> in <code>compose</code>	Malware persistence
Drop all capabilities	<code>cap_drop: ALL</code> in <code>compose</code>	Privilege escalation
No new privileges	<code>no-new-privileges:true</code> <code>seccomp</code>	Setuid binary abuse
dumb-init PID 1	<code>ENTRYPOINT ["dumb-init", "-"]</code>	Signal handling, zombie processes
Health check	<code>HEALTHCHECK</code> in <code>Dockerfile</code>	Zombie container detection
Resource limits	CPU/memory limits in <code>deploy block</code>	Denial of Service
Secrets from env	<code>env_file: .env</code>	Credential leakage in images
Private network	Internal bridge network	Lateral movement

Table 6.1: CIS Docker Benchmark controls implemented

6.1.2 Problems Encountered During Docker Setup

Problem 1 — Build context missing files: The Dockerfile expected `requirements.txt`, `app_secure.py`, and `templates/` in the build context, but these files were in `part1/` not `part3/`.

Fix:

```
1 cp ../part1/app_secure.py .
2 cp ../requirements.txt .
3 cp -r ../part1/templates .
4 # Change build context in docker-compose.yml
5 sed -i 's|context: ../part1|context: .|g' docker-compose.yml
```

Problem 2 — docker-compose.yml references wrong paths: The original file referenced `../session1` and `../session3` which do not exist.

Fix:

```
1 sed -i 's|../session1|../part1|g' docker-compose.yml
2 sed -i 's|../session3|../part3|g' docker-compose.yml
```

Problem 3 — Custom seccomp profile not supported on WSL2: The compose file referenced `seccomp_profile.json` which caused a `bind-mount /proc/ns/net` error on WSL2.

Fix: Remove the seccomp line from `docker-compose.yml`:

```
1 sed -i '/seccomp:seccomp_profile.json/d' docker-compose.yml
```

Note: In a production Linux environment, the custom seccomp profile would restrict available system calls, further reducing the container attack surface.

Problem 4 — gunicorn ModuleNotFoundError: The multi-stage build copied packages from `/root/.local` (builder stage) to `/home/appuser/.local` (production stage), but gunicorn could not find its own modules at runtime.

Root cause: Python packages installed as `root` in builder stage are copied to `appuser` home, but the internal module references still point to the original `root` paths.

Fix: Run `security_monitoring.py` directly as a standalone WAF on port 5002, and use Flask's built-in server for `app_secure.py` during testing. In production, use a single-stage build with system-wide pip install.

6.1.3 Building the Docker Image

```
1 docker build -t lab-app:v1 -f Dockerfile .
```

Listing 6.1: Building the hardened Docker image

6.1.4 Trivy Container Scan

```
1 trivy image lab-app:v1 --severity HIGH,CRITICAL
```

Listing 6.2: Running Trivy container vulnerability scan

Trivy found **7 HIGH** vulnerabilities, **0 CRITICAL**:

Library	CVE	Severity	Description
libncursesw6	CVE-2025-69720	HIGH	Buffer overflow → arbitrary code execution
libtinfo6	CVE-2025-69720	HIGH	Same ncurses vulnerability
ncurses-base	CVE-2025-69720	HIGH	Same ncurses vulnerability
ncurses-bin	CVE-2025-69720	HIGH	Same ncurses vulnerability
jaraco.context	CVE-2026-23949	HIGH	Path traversal via malicious tar archives
wheel	CVE-2026-24049	HIGH	Privilege escalation via malicious wheel file
wheel (vendor)	CVE-2026-24049	HIGH	Same wheel vulnerability

Table 6.2: Trivy scan results — 7 HIGH, 0 CRITICAL

lab-app:v1 (debian 13.5)						
Total: 4 (HIGH: 4, CRITICAL: 0)						
Library	Vulnerability	Severity	Status	Installed Version	Fixed Version	Title
libncursesw6	CVE-2025-69720	HIGH	affected	6.5+20250216-2		ncurses: ncurses: Buffer overflow vulnerability may lead to arbitrary code execution. https://avd.aquasec.com/nvd/cve-2025-69720
libtinfo6						
ncurses-base						
ncurses-bin						
Python (python-pkg)						
Total: 3 (HIGH: 3, CRITICAL: 0)						
Library	Vulnerability	Severity	Status	Installed Version	Fixed Version	Title
jaraco.context (METADATA)	CVE-2026-23949	HIGH	fixed	5.3.0	6.1.0	jaraco.context: jaraco.context: Path traversal via malicious tar archives https://avd.aquasec.com/nvd/cve-2026-23949
wheel (METADATA)	CVE-2026-24049			0.45.1	0.46.2	wheel: wheel: Privilege Escalation or Arbitrary Code Execution via malicious wheel file... https://avd.aquasec.com/nvd/cve-2026-24049

Figure 6.1: Trivy container vulnerability scan results

Note: The 4 ncurses vulnerabilities have no fix available yet (**Status:** `affected`). The wheel and jaraco.context vulnerabilities have fixes available — upgrade to `wheel>=0.46.2` and `jaraco.context>=6.1.0`. No CRITICAL vulnerabilities were found, confirming the hardened image has a minimal attack surface.

6.1.5 Hardening Verification

```

1 # Verify non-root user
2 docker run --rm lab-app:v1 whoami
3 # Expected output: appuser
4
5 # Verify read-only filesystem
6 docker run --rm --read-only lab-app:v1 touch /test
7 # Expected output: Read-only file system
8
```

```

9 # Verify security options via docker inspect
10 docker inspect lab-app:v1 | grep -A5 "User"
11 # Expected output: "User": "appuser"
12
13 # Verify after compose
14 docker inspect lab_web | grep -A10 "SecurityOpt"
15 # Expected output: "no-new-privileges:true"

```

Listing 6.3: Verifying CIS Docker Benchmark controls

```

(nigo@DESKTOP-V9JBRA4)-[/mnt/c/Users/ELECTRO-RIMO/Desktop/bureau/9raya/DevSecOps/webSecurity/dev-sec-ops-lab-1/pa
rt3]
$ docker run --rm lab-app:v1 whoami
appuser

(nigo@DESKTOP-V9JBRA4)-[/mnt/c/Users/ELECTRO-RIMO/Desktop/bureau/9raya/DevSecOps/webSecurity/dev-sec-ops-lab-1/pa
rt3]
$ docker run --rm --read-only lab-app:v1 touch /test
touch: cannot touch '/test': Read-only file system

(nigo@DESKTOP-V9JBRA4)-[/mnt/c/Users/ELECTRO-RIMO/Desktop/bureau/9raya/DevSecOps/webSecurity/dev-sec-ops-lab-1/pa
rt3]
$ docker inspect lab-app:v1 | grep -A5 "User"
    "User": "appuser",
    "ExposedPorts": {
      "8080/tcp": {}
    },
    "Env": [
      "PATH=/home/appuser/.local/bin:/usr/local/bin:/usr/local/sbin:/usr/bin:/sbin:/bin",

```

Figure 6.2: Non-root user and read-only filesystem verified

```

(nigo@DESKTOP-V9JBRA4)-[/mnt/c/Users/ELECTRO-RIMO/Desktop
rt3]
$ docker inspect lab_web | grep -A10 "SecurityOpt"
    "SecurityOpt": [
      "no-new-privileges:true"
    ],
    "Tmpfs": {
      "/app/data": "size=100m",
      "/tmp": "size=50m,mode=1777"
    },
    "UTSMode": "",
    "UsernsMode": "",
    "ShmSize": 67108864,
    "Runtime": "runc",

```

Figure 6.3: Security options confirmed via docker inspect

6.1.6 Docker Compose Deployment

```

1 docker compose up -d
2 docker ps

```

Listing 6.4: Starting the hardened stack

```
(nigo@DESKTOP-V9JBRA4)-[~/mnt/c/Users/ELECTRO-RIMO/Desktop/bureau/9raya/DevSecOps/webSecurity/dev-sec-ops-lab-1/part3]
$ docker compose up -d
WARN[0000] /mnt/c/Users/ELECTRO-RIMO/Desktop/bureau/9raya/DevSecOps/webSecurity/dev-sec-ops-lab-1/part3/docker-compos
e.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
[*] up 4/4
✓Network part3_app_network Created 0.1s
✓Container lab_redis Started 1.5s
✓Container lab_web Started 1.8s
✓Container lab_nginx Started 2.6s
```

Figure 6.4: Hardened Docker Compose stack running

6.2 Runtime Security Monitoring — WAF

What is Runtime WAF?

A Web Application Firewall (WAF) inspects incoming HTTP requests at runtime and blocks malicious payloads before they reach the application. Unlike SAST (which scans code) and DAST (which tests a running app), a WAF provides *continuous* protection in production.

The `security_monitoring.py` module implements a Flask middleware WAF with:

- Pattern-based attack detection (SQL injection, XSS, command injection)
- Rate limiting (blocks after 30 requests/minute per IP)
- Structured JSON security event logging (SIEM-compatible)
- Defensive HTTP security headers on every response

6.2.1 Starting the WAF

Problem encountered: `security_monitoring.py` uses `request.remote_addr` for IP detection, which always returns `127.0.0.1` for local requests — making it impossible to distinguish between different clients in testing.

Fix: Added `X-Forwarded-For` header support:

```
1 # Before
2 ip = request.remote_addr or "unknown"
3
4 # After
5 ip = request.headers.get("X-Forwarded-For",
6     request.remote_addr) or "unknown"
```

This allows spoofing a specific attacker IP during incident response simulation.

```
1 cd part3
2 source ../.venv/bin/activate
3 python security_monitoring.py 2>&1 | tee security_events.log
```

Listing 6.5: Starting the WAF middleware

```

Security monitoring active on http://127.0.0.1:5002
Try: curl 'http://127.0.0.1:5002/api/demo?q=<script>alert(1)</script>'
Try: curl 'http://127.0.0.1:5002/api/demo?q=' OR 1=1--'
* Serving Flask app 'security_monitoring'
* Debug mode: off
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI
server instead.
* Running on http://127.0.0.1:5002
INFO:werkzeug:Press CTRL+C to quit

```

Figure 6.5: WAF middleware started on port 5002

6.2.2 WAF Rule Testing

```

1 # SQL Injection detection
2 curl 'http://127.0.0.1:5002/api/demo?q=SELECT**FROM+users'
3 # Response: {"error":"Request blocked by security policy"}
4
5 # XSS detection
6 curl "http://127.0.0.1:5002/api/demo?q=<script>alert(1)</script>
  >"
7 # Response: {"error":"Request blocked by security policy"}
8
9 # Command injection detection
10 curl 'http://127.0.0.1:5002/api/demo?q=;+id'
11 # Response: {"error":"Request blocked by security policy"}
12
13 # Legitimate request (should pass)
14 curl 'http://127.0.0.1:5002/api/demo?q=hello'
15 # Response: 200 OK

```

Listing 6.6: Testing WAF detection rules

Payload	Rule	Description	Response	Severity
SELECT * FROM users	WAF-001	SQL In- jection	400 Blocked	HIGH
<script>alert(1)</script>	WAF-002	XSS	400 Blocked	HIGH
; id	WAF-004	Command Injection	400 Blocked	CRITICAL
hello	—	Legitimate request	200 Allowed	—
35 req/min	Rate Limit	Excessive requests	429 Blocked	MEDIUM

Table 6.3: WAF rule testing results

```

(.venv)(nigo@DESKTOP-V9JBRA4) - [mnt/c/Users/ELECTRO-RIMO/Desktop/bureau/9raya/DevSecOps/webSecurity/dev-sec-ops-lab-1/part3]
└─$ curl http://127.0.0.1:5002/api/demo?q=SELECT**FROM+users
{"error":"Request blocked by security policy"}

(.venv)(nigo@DESKTOP-V9JBRA4) - [mnt/c/Users/ELECTRO-RIMO/Desktop/bureau/9raya/DevSecOps/webSecurity/dev-sec-ops-lab-1/part3]
└─$ curl "http://127.0.0.1:5002/api/demo?q=<script>alert(1)</script>"
{"error":"Request blocked by security policy"}

(.venv)(nigo@DESKTOP-V9JBRA4) - [mnt/c/Users/ELECTRO-RIMO/Desktop/bureau/9raya/DevSecOps/webSecurity/dev-sec-ops-lab-1/part3]
└─$ curl 'http://127.0.0.1:5002/api/demo?q=;id'
{"error":"Request blocked by security policy"}

(.venv)(nigo@DESKTOP-V9JBRA4) - [mnt/c/Users/ELECTRO-RIMO/Desktop/bureau/9raya/DevSecOps/webSecurity/dev-sec-ops-lab-1/part3]
└─$ curl 'http://127.0.0.1:5002/api/demo?q=hello'
{"echo":"hello","length":5}

(.venv)(nigo@DESKTOP-V9JBRA4) - [mnt/c/Users/ELECTRO-RIMO/Desktop/bureau/9raya/DevSecOps/webSecurity/dev-sec-ops-lab-1/part3]
└─$ for i in $(seq 1 35); do curl -s http://127.0.0.1:5002/ > /dev/null; done
curl http://127.0.0.1:5002/
{"error":"Too many requests"}

```

Figure 6.6: WAF blocking attack payloads in real time

6.2.3 Security Headers

```
1 curl -I http://127.0.0.1:5002/
```

Listing 6.7: Verifying security headers with curl

Header	Value	Attack Prevented
X-Content-Type-Options	nosniff	MIME sniffing attacks
X-Frame-Options	DENY	Clickjacking
X-XSS-Protection	1; mode=block	Reflected XSS (legacy browsers)
Content-Security-Policy	default-src 'self'	XSS, data injection

Table 6.4: Security headers implemented by WAF middleware

```

(.venv)(nigo@DESKTOP-V9JBRA4) - [mnt/c/Users/ELECTRO-RIMO/Desktop/bureau/9raya/DevSecOps/webSecurity/dev-sec-ops-lab-1/part3]
└─$ curl -I http://127.0.0.1:5002/
HTTP/1.1 200 OK
Server: Werkzeug/3.0.3 Python/3.13.12
Date: Wed, 20 May 2026 20:27:16 GMT
Content-Type: application/json
Content-Length: 91
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
X-XSS-Protection: 1; mode=block
Content-Security-Policy: default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' data:; frame-ancestors 'none';
Connection: close

```

Figure 6.7: Security headers confirmed on normal request

Connection to ZAP findings: In Part 2, OWASP ZAP detected 6 missing security headers on the vulnerable app. The WAF middleware in Part 3 adds all of them — X-Frame-Options, X-Content-Type-Options, Content-Security-Policy, and X-XSS-Protection — demonstrating the complete defense-in-depth cycle from detection to remediation.

6.3 Incident Response Exercise

6.3.1 Scenario — P1 Active SQL Injection Attack

Scenario: 50 WAF_BLOCK HIGH severity alerts received from IP 203.0.113.42 in under 2 seconds. The attacker is targeting the /login endpoint with automated SQL injection payloads.

6.3.2 Step 1 — Identify the Attacker IP from Structured Logs

The attack was simulated by sending 50 SQL injection requests with a spoofed IP header:

```
1 for i in $(seq 1 50); do
2   curl -s 'http://127.0.0.1:5002/login?username=admin%27+OR
3     +%271%27%3D%271' \
4   -H "X-Forwarded-For: 203.0.113.42" > /dev/null
5 done
```

Listing 6.8: Simulating the attack

The attacker IP was identified by parsing the structured JSON logs:

```
1 grep "WAF_BLOCK" security_events.log | python3 -c "
2 import sys, json
3 for line in sys.stdin:
4     if 'WAF_BLOCK' in line:
5         start = line.find('{')
6         if start != -1:
7             event = json.loads(line[start:])
8             print(f'Time: {event["timestamp"]} | IP: {event["
9               source_ip"]} | Rule: {event["details"]["rule_name"]}')
10 " | head -5
```

Listing 6.9: Identifying attacker from WAF logs

Result:

```
Time: 2026-05-23T16:52:33.716761Z | IP: 203.0.113.42 | Rule: SQL Injection
Time: 2026-05-23T16:52:33.741814Z | IP: 203.0.113.42 | Rule: SQL Injection
Time: 2026-05-23T16:52:33.760351Z | IP: 203.0.113.42 | Rule: SQL Injection
...
Total: 30 WAF_BLOCK events from 203.0.113.42 in < 2 seconds
```



```

Time: 2026-05-23T16:52:33.716761Z | IP: 203.0.113.42 | Rule: SQL Injection | Path: /login
Time: 2026-05-23T16:52:33.741814Z | IP: 203.0.113.42 | Rule: SQL Injection | Path: /login
Time: 2026-05-23T16:52:33.760351Z | IP: 203.0.113.42 | Rule: SQL Injection | Path: /login
Time: 2026-05-23T16:52:33.777892Z | IP: 203.0.113.42 | Rule: SQL Injection | Path: /login
Time: 2026-05-23T16:52:33.797650Z | IP: 203.0.113.42 | Rule: SQL Injection | Path: /login
Time: 2026-05-23T16:52:33.821627Z | IP: 203.0.113.42 | Rule: SQL Injection | Path: /login
Time: 2026-05-23T16:52:33.842272Z | IP: 203.0.113.42 | Rule: SQL Injection | Path: /login
Time: 2026-05-23T16:52:33.861875Z | IP: 203.0.113.42 | Rule: SQL Injection | Path: /login
Time: 2026-05-23T16:52:33.880802Z | IP: 203.0.113.42 | Rule: SQL Injection | Path: /login
Time: 2026-05-23T16:52:33.898370Z | IP: 203.0.113.42 | Rule: SQL Injection | Path: /login
Time: 2026-05-23T16:52:33.917701Z | IP: 203.0.113.42 | Rule: SQL Injection | Path: /login
Time: 2026-05-23T16:52:33.937445Z | IP: 203.0.113.42 | Rule: SQL Injection | Path: /login
Time: 2026-05-23T16:52:33.957032Z | IP: 203.0.113.42 | Rule: SQL Injection | Path: /login
Time: 2026-05-23T16:52:33.974352Z | IP: 203.0.113.42 | Rule: SQL Injection | Path: /login
Time: 2026-05-23T16:52:33.991618Z | IP: 203.0.113.42 | Rule: SQL Injection | Path: /login
Time: 2026-05-23T16:52:34.009963Z | IP: 203.0.113.42 | Rule: SQL Injection | Path: /login
Time: 2026-05-23T16:52:34.026665Z | IP: 203.0.113.42 | Rule: SQL Injection | Path: /login
Time: 2026-05-23T16:52:34.043168Z | IP: 203.0.113.42 | Rule: SQL Injection | Path: /login
Time: 2026-05-23T16:52:34.062103Z | IP: 203.0.113.42 | Rule: SQL Injection | Path: /login
Time: 2026-05-23T16:52:34.084280Z | IP: 203.0.113.42 | Rule: SQL Injection | Path: /login
Time: 2026-05-23T16:52:34.100134Z | IP: 203.0.113.42 | Rule: SQL Injection | Path: /login
Time: 2026-05-23T16:52:34.120107Z | IP: 203.0.113.42 | Rule: SQL Injection | Path: /login
Time: 2026-05-23T16:52:34.136924Z | IP: 203.0.113.42 | Rule: SQL Injection | Path: /login
Time: 2026-05-23T16:52:34.152695Z | IP: 203.0.113.42 | Rule: SQL Injection | Path: /login
Time: 2026-05-23T16:52:34.167867Z | IP: 203.0.113.42 | Rule: SQL Injection | Path: /login
Time: 2026-05-23T16:52:34.182905Z | IP: 203.0.113.42 | Rule: SQL Injection | Path: /login
Time: 2026-05-23T16:52:34.197347Z | IP: 203.0.113.42 | Rule: SQL Injection | Path: /login
Time: 2026-05-23T16:52:34.213038Z | IP: 203.0.113.42 | Rule: SQL Injection | Path: /login

```

Figure 6.8: Attacker IP identified from structured WAF logs

6.3.3 Step 2 — Block the Attacker IP

```

1 # Block IP using iptables
2 sudo iptables -I INPUT -s 203.0.113.42 -j DROP
3
4 # Verify rule was added
5 sudo iptables -L INPUT -n | head -5

```

Listing 6.10: Blocking attacker at infrastructure level

Result:

Chain INPUT (policy ACCEPT)

target	prot	opt	source	destination
DROP	all	—	203.0.113.42	0.0.0.0/0

```

(.venv)(nigo@DESKTOP-V9JBRA4)-[~/mnt/c/Users/ELECTRO-RIMO/Desktop/bureau/9raya/DevSecOps/webSecurity/dev-sec-ops-lab-1/part3]
$ sudo iptables -L INPUT -n | head -10
Chain INPUT (policy ACCEPT)
target prot opt source destination
DROP all -- 203.0.113.42 0.0.0.0/0

```

Figure 6.9: Attacker IP blocked at infrastructure level via iptables

6.3.4 Step 3 — Rotate the SECRET_KEY

```

1 export SECRET_KEY=$(python -c "import secrets; print(secrets.
   token_hex(32))")
2 echo "New SECRET_KEY: $SECRET_KEY"

```

Listing 6.11: Rotating SECRET_KEY without downtime

Result:

New SECRET_KEY: dc75d2f9e910eeb7236e39bdd778e27cab2a3cdb260295d44a348e78

```

(.venv)(nigo@DESKTOP-V9JBRA4)-[/mnt/c/Users/ELECTRO-RIMO/Desktop/bureau/9raya/DevSecOps-1/part3]
$ echo "New SECRET_KEY: $SECRET_KEY"
New SECRET_KEY: dc75d2f9e910eeb7236e39bdd778e27cab2a3cdb260295d44a348e781a7f6bf0

```

Figure 6.10: SECRET_KEY rotated as precautionary measure

6.3.5 Step 4 — Incident Documentation

Field	Details
Incident ID	IR-2026-001
Date	2026-05-23
Severity	P1 — HIGH
Status	RESOLVED
First alert	2026-05-23T16:52:33Z
Last alert	2026-05-23T16:52:34Z
Duration	< 2 seconds (automated attack)
Attacker IP	203.0.113.42
Target endpoint	/login
Attack type	SQL Injection (WAF-001)
Payload	admin' OR '1'='1
Total attempts	30 blocked by WAF
Action 1	Attacker IP identified from structured WAF logs
Action 2	IP blocked: <code>iptables -I INPUT -s 203.0.113.42 -j DROP</code>
Action 3	SECRET_KEY rotated: <code>secrets.token_hex(32)</code>
Action 4	Incident documented in <code>incident_report.txt</code>

Table 6.5: Incident response documentation

6.3.6 Step 5 — Post-Mortem Recommendations

ID	Recommendation
R1	Implement automatic IP blocking after 10 WAF_BLOCK events from the same IP within 60 seconds
R2	Integrate WAF logs with a SIEM (Splunk/Elastic/Datadog) for real-time alerting and dashboards
R3	Add CAPTCHA to the /login endpoint to prevent automated credential stuffing
R4	Implement account lockout after 5 consecutive failed login attempts
R5	Deploy an IP reputation service to proactively block known malicious IP ranges
R6	Store SECRET_KEY in a secrets vault (HashiCorp Vault / AWS SSM) — never in environment variables
R7	Enable fail2ban to automate IP blocking based on WAF log patterns

Table 6.6: Post-mortem security recommendations

7. Conclusion

This lab provided a comprehensive hands-on journey through the complete DevSecOps lifecycle — from identifying and exploiting vulnerabilities, to automating security testing, to hardening a production deployment and responding to live attacks.

7.1 Summary of Achievements

Part 1 — Build & Break demonstrated that the most critical web vulnerabilities are not theoretical — they are trivially exploitable in real applications. SQL injection bypassed authentication in seconds, XSS executed arbitrary JavaScript in the victim's browser, command injection gave full OS access, and path traversal exposed the application's own source code. Each vulnerability was then remediated in `app_secure.py` following the principle of **never trusting user input**.

Part 2 — Test & Automate proved that manual security testing is not enough. Bandit eliminated 100% of HIGH and MEDIUM severity findings automatically. Safety identified 10 real CVEs in production dependencies. Gitleaks detected hardcoded secrets in git history. OWASP ZAP revealed missing security headers. GitHub Actions tied all these tools into a CI/CD pipeline that **blocks insecure code from being merged** — making security a continuous, automated practice rather than a one-time gate.

Part 3 — Harden & Monitor showed that application-level security is not enough on its own. Docker hardening following CIS Benchmark guidelines ensures that even a compromised container cannot escalate privileges or persist malware. The WAF middleware provided runtime protection, detecting and blocking SQL injection, XSS, and command injection payloads in real time. The incident response exercise demonstrated a complete P1 response workflow: detect, contain, rotate credentials, document, and prevent recurrence.

7.2 Key Lessons Learned

Lesson	Detail
Never trust user input	Every vulnerability in Part 1 came from unsanitized input
Shift security left	Catching bugs at commit time costs far less than in production
Defense in depth	No single control is sufficient — layer SAST, SCA, WAF, hardening
False positives are real	Gitleaks, ZAP, and pytest all produced false positives requiring tuning
Environment matters	WSL2 behaves differently from production Linux — test on real hardware
Secrets must be vaulted	Hardcoded secrets are found instantly by automated scanners
Logs enable response	Structured JSON logs made attacker identification trivial

Table 7.1: Key security lessons from the lab

7.3 DevSecOps Workflow Completed

The complete DevSecOps cycle was demonstrated end-to-end:

1. **Model** — STRIDE threat modeling identified 8 threats before writing any code
2. **Code** — Secure implementation applied parameterized queries, `safe_join`, template escaping
3. **Scan** — Bandit, Safety, and Gitleaks caught vulnerabilities and secrets automatically
4. **Test** — pytest security tests validated that attacks are blocked by the secure app
5. **Release** — GitHub Actions pipeline enforced security gates on every push
6. **Deploy** — CIS-hardened Docker container minimized the production attack surface
7. **Monitor** — WAF middleware detected and blocked live attacks at runtime
8. **Respond** — Incident response runbook handled a P1 SQL injection attack in under 5 minutes

7.4 Known Limitations and Future Work

Several limitations were encountered during the lab that would need to be addressed in a real production environment:

- **WSL2 constraints:** Custom seccomp profiles, network namespace isolation, and some Docker security options are not fully supported on WSL2. A production deployment on native Linux would enable these controls.
- **unicorn path issue:** The multi-stage Dockerfile's package path resolution failed on WSL2. A production fix would use a single-stage build with system-wide installation or a properly configured virtual environment.
- **WAF as middleware:** `security_monitoring.py` runs as a standalone app rather than integrated middleware. A production WAF should be integrated at the nginx/reverse proxy level for zero-downtime enforcement.
- **CSRF protection:** The secure app does not implement CSRF tokens. A production Flask app should use `Flask-WTF` for CSRF protection on all form endpoints.
- **HTTPS:** The lab ran on HTTP. All production deployments must enforce HTTPS with valid TLS certificates and HSTS headers.

7.5 Final Remarks

This lab reinforced that security is not a product — it is a continuous process. The tools and techniques demonstrated here reflect real-world industry practices used by security engineering teams at major organizations. The complete pipeline from threat modeling to incident response, automated and reproducible, is the foundation of a mature DevSecOps program.

Repository: The complete lab code, fixed pipeline, and this report are available at:

<https://github.com/Mossaab49/DevSecOps>

All problems encountered during the lab, along with their exact fixes, are documented in this report to help future students avoid the same issues.

References

1. OWASP Top Ten. *OWASP Foundation*. <https://owasp.org/www-project-top-ten/>
2. STRIDE Threat Modeling. *Microsoft Security*. <https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool-threats>
3. Bandit Documentation. *PyCQA*. <https://bandit.readthedocs.io/>
4. Safety Documentation. <https://pyup.io/safety/>
5. CIS Docker Benchmark v1.6. *Center for Internet Security*.
6. Werkzeug Documentation — safe_join. <https://werkzeug.palletsprojects.com/>
7. Flask Security Documentation. <https://flask.palletsprojects.com/en/latest/security/>
8. Lab Repository. <https://github.com/said-dev-ops/dev-sec-ops-lab-1>